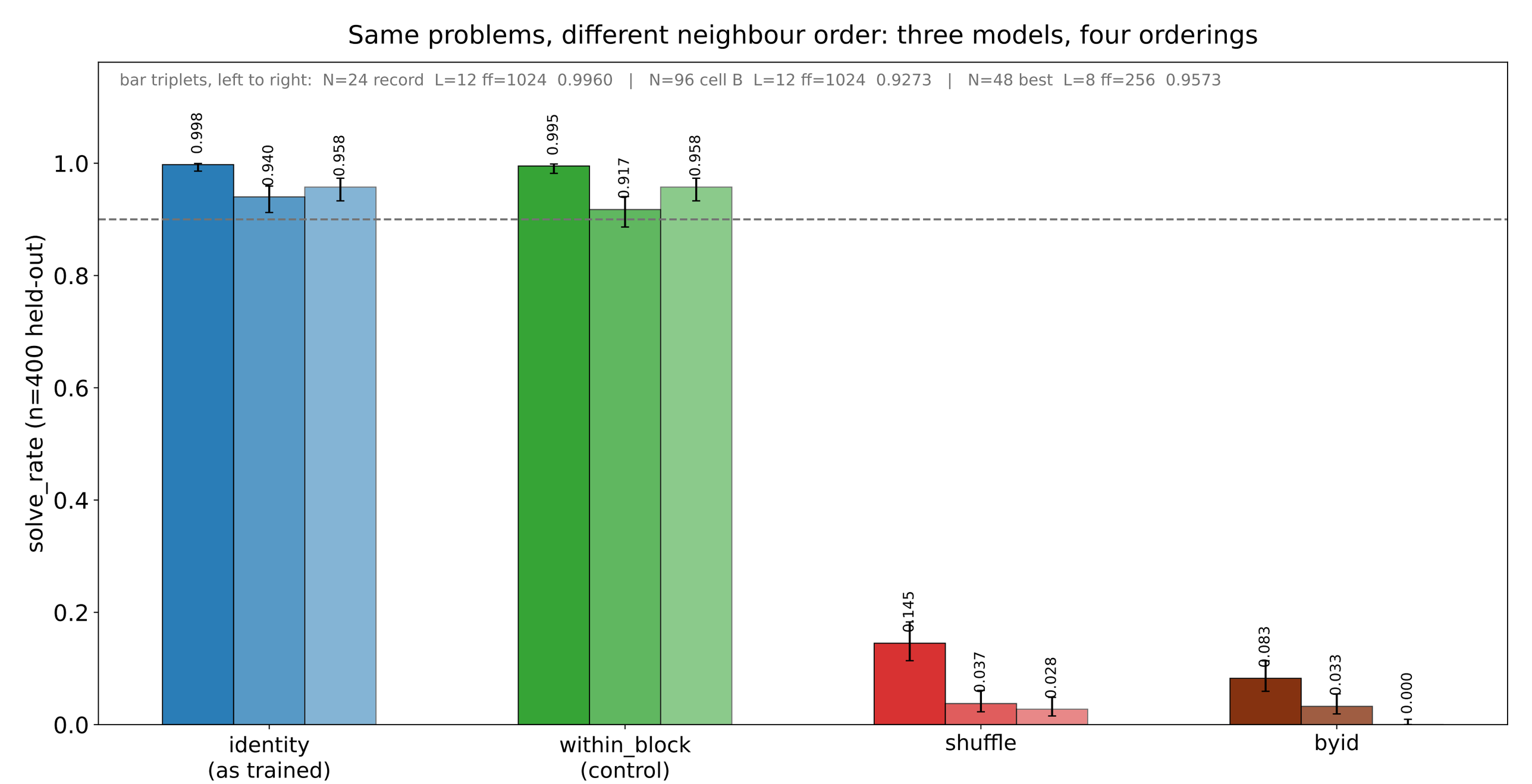
The networks were not solving graph colouring

They were reading the planted colours off the token order

Permuting each node's neighbour list -- same edges, same colouring, same hints, same DD trace, all asserted equal -- drops every checkpoint in the project from 0.94-1.00 to 0.00-0.15.

job 49655447 · eval-only, 18 min · 2026-10-26

The in-distribution control (permute only within a colour group) ties identity on all three models, so the collapse is the colour code, not distribution shift.


Same problems, different neighbour order: three models, four orderings
bar triplets, left to right: N=24 record L=12 ff=1024 0.9960 | N=96 cell B L=12 ff=1024 0.9273 | N=48 best L=8 ff=256 0.9573
solve_rate (n=400 held-out)
0.0
0.2
0.4
0.6
0.8
1.0
0.998
0.940
0.958
0.995
0.917
0.958
0.145
0.037
0.028
0.083
0.033
0.000
identity
(as trained)
within_block
(control)
shuffle
byid
Each ordering preserves the edge set per node, the colouring, the hint set and the DD trace -- the trace is recomputed
on the reordered adjacency and asserted equal, so these are the same instances asked the same question. Error bars are
Wilson 95%.

Paired, on the same 400 instances

model	ordering	identity	reordered	delta	paired p	well-formed
N=24 record	within_block	0.9975	0.9950	-0.0025	1	1.000
	shuffle	0.9975	0.1450	-0.8525	4.46e-103	0.848
	byid	0.9975	0.0825	-0.9150	1.33e-110	0.775
N=96 cell B	within_block	0.9400	0.9175	-0.0225	0.175	0.958
	shuffle	0.9400	0.0375	-0.9025	4.26e-109	0.185
	byid	0.9400	0.0325	-0.9075	1.06e-109	0.180
N=48 best	within_block	0.9575	0.9575	+0.0000	1	0.995
	shuffle	0.9575	0.0275	-0.9300	2.08e-112	0.695
	byid	0.9575	0.0000	-0.9575	1.02e-115	0.672

GREEN = within_block, the design's own sanity check: it preserves the colour code and changes only the order, and it ties identity on all three models. That is what licenses reading the RED rows as the colour code rather than as distribution shift. McNemar exact on discordant pairs. Well-formed is the fraction of outputs that parse at all: its collapse at N=96 (0.185) means the model stops producing a valid trace, not merely a wrong one.

Where the leak comes from, and the fix

planted_regular_graph (sudoku_dd.py:49) appends edges in class-pair block order

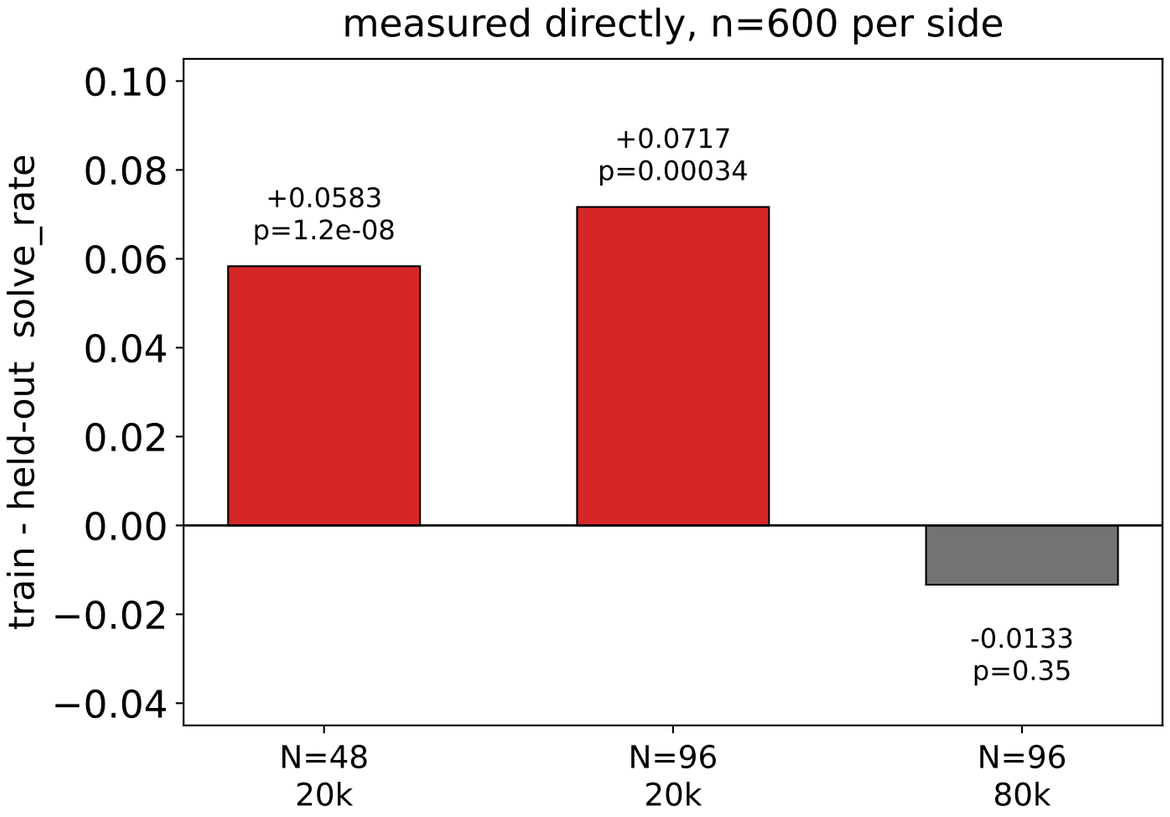
```
for a in range(q):
    for b in range(a + 1, q):   # (0,1), (0,2), (0,3), (1,2), ...
        edge_blocks.append(...)
...
for x, y in edges:   # adj filled in that order
    adj[x, fill[x]] = y
```

The node relabelling that follows shuffles node IDs but not the order edges were appended, so adj[v] arrives sorted by the neighbour's TRUE COLOUR -- and encode_instance emits it raw.

measurement	N=24	N=48	N=96
adj[v] sorted by neighbour colour	**100.0%**	**100.0%**	**100.0%**
the one absent colour IS v's own	67.1%	61.3%	--
round events in ascending node index	100.0%	100.0%	100.0%
within-colour-block order sorted by id	--	38.9%	36.2%
AFTER THE FIX (shuffle_adj=True)	**5.7%**	**9.1%**	**--**

Within-colour-block order is already arbitrary at chance (36-39% sorted), which is why within_block is in-distribution and makes a valid control. The fix permutes adj[v] in build_instances from a dedicated rng stream: colour-sorted 100.0% -> 5.7%/9.1% (chance for d=5 over q=4), the pre-fix generator stream stays bit-identical, and the edge set, colouring, hints and DD trace are unchanged instance by instance.

The other question: are they memorizing their training set?



run	train	held-out	gap	p
N=48 L=12 @20000	**0.9967**	**0.9383**	**+0.0583**	**1.2e-08**
N=96 ctrl @20000	**0.8967**	**0.8250**	**+0.0717**	**3.4e-04**
N=96 cell B @80000	**0.9267**	**0.9400**	**-0.0133**	**0.355**

Yes at 20000, and data closes it: no gap at 80000.

Pre-registered in advance and CONFIRMED -- the training set is built once, before the epoch loop, so a 150-epoch run sees the same instances 150 times and can memorize them.

WITHDRAWN: the train-val NLL gaps I had been reading this off. train_nll is averaged over the epoch WHILE weights update, so it carries a +0.3263 bias at epoch 10 (N=96 control, decaying). The sign of a +-0.03 gap means nothing, and finding 15's MECHANISM rested on +0.0239. Its +0.1007 result is unaffected.

Training index i is reproducible as build_instances(cfg, 1, i), so a prefix of the training set is recoverable exactly and the model can be scored on the instances it actually saw. No run in this project had ever been scored that way. Unpaired two-proportion z-test: the train sample and the held-out sample are different instances.

What this costs, and what is running now

result	survives?	why
solve_rate ladder 0.9960 / 0.9573 / 0.9273	**AS DECODING**	**real on this generator, not deduction**
finding 2: the model is not running DD	**STRENGTHENED**	**now with a mechanism for why it need not**
L*(T), L*(N), the depth-vs-N series	**RE-MEASURE**	**depth to decode, not depth to deduce**
finding 13 / N=48 thin-deep beats wide	**RE-MEASURE**	**a claim about decoding capacity**
finding 15: data is the binding lever (+0.1007)	RESULT HOLDS	mechanism withdrawn, see page 5
memorization closes with data	**NEW, CLEAN**	**measured on instances, not on NLL**

49659502 -- the first run in this project measuring deduction

One change from the record model: --shuffle-adj. N=24, L=12, ff=1024, 80000 instances, 300 epochs.
Pre-registered: below 0.9960 at p<0.05 but above 0.50 -- a real cost, not a collapse; order-insensitive
across all colour-free orderings; and re-instating the leak (bycolor) does NOT help it. ~6 h.

RE-MEASURE does not mean wrong: those runs really do score what they score, on instances drawn from this generator. It means the quantity they measure is the depth and width needed to decode a positional code, not to execute T rounds of deduction, so the reasoning-depth reading of them does not go through. Which of them to redo is a scoping call -- 49659502 is one cheap cell that makes that call informed rather than a guess.